

Forzar una salida estructurada, por ejemplo JSON

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 8 de 12 · Notas en español (es-ES)

Si solo decís «dame JSON», Claude suele devolver JSON *más* un párrafo amable («Aquí está el JSON, ¿quieres algo más?»). Entonces `json.loads` peta. El patrón del 19 de agosto fue **ingeniería de prompts específica de Claude**: instrucción de usuario que nombra el esquema, un **prefill de assistant** que abre un bloque JSON vallado, y una **secuencia de parada** en la valla de cierre para que la generación se quede dentro del objeto.

El modo de fallo

Los modelos de lenguaje natural quieren ser corteses. La prosa al principio o al final es mortal si un servicio espera un solo objeto con cadenas entre comillas (ejemplo de Vijay: «name: Vijay»). Los consumidores estructurados — agentes que escriben archivos, generadores de config AWS, APIs — necesitan **solo** la carga.

El prefill es una semilla, no un segundo paso

El prefill de aula era un mensaje extra: `role: assistant`, contenido que empieza con una valla markdown JSON (en voz: ticks + `json`). El mensaje de usuario era p. ej. «Lista 5 lenguajes de programación populares y sus casos de uso en JSON». El modelo continúa desde la valla, así que lo siguiente esperado es { ... La secuencia de parada es la valla de cierre. Mritunjay preguntó si el servidor genera todo y luego «traduce» o trunca: **no** — así se monta el prompt para que la decodificación empiece en la semilla. Los tokens facturados son la continuación, no un preámbulo descartado.

Varios lenguajes vallados en una respuesta

Un demo más duro pedía una muestra de configuración de usuario en **bash**, **XML** y **JSON**, cada uno en su bloque, **sin explicaciones entre bloques**, acabando en un marcador tipo END que también iba en `stop_sequences`. Es instrucción con forma de few-shot: mostrad las etiquetas de valla exactas. Los agentes que deben escribir archivos adoran este contrato.

Técnica	Fortaleza	Debilidad
Solo instrucción («responde con JSON»)	Simple	Se cuela prosa
Prefill + stop en vallas	Funciona sin modo esquema; se enseñó en clase	Frágil si el modelo cambia de valla o añade claves
Structured outputs / JSON schema oficial	Decodificación acotada si el modelo lo soporta	Hay que cuadrar la API vigente; igualmente validar
Pydantic / JSON Schema en vuestro código	Valida tipos a posteriori	Por sí solo no corta la prosa salvo que lo combinéis

Validad la forma en código, no siempre con otro LLM

Más tarde el mismo día, evals: si el contrato es «objeto JSON con name: string y age: number», un modelo **Pydantic** (o similar) basta para la forma. Un LLM-as-judge es exceso y caro. Tools o recuperación juzgan *valores* (¿esta edad es correcta en RR. HH.?). Mantened esas capas distintas.

Qué poner en el contenido de usuario

- Nombrad el formato y las claves.
- Prohibid claves extra / markdown si no usáis vallas.
- Dad un ejemplo compacto (few-shot) si la forma está anidada.
- Dejad temperature baja para JSON de extracción.

PRODUCCIÓN 2026

Truco de clase = prefill + stop. La guía actual de Anthropic documenta también **structured outputs**. En entrega UST, preferid APIs acotadas por esquema cuando el modelo las soporte, y seguid parseando. Examen: sabed explicar el patrón de prefill que programó Vijay.

XML como sobre alternativo

Si la organización ya piensa en XML, las vallas pueden ser etiquetas XML en vez de markdown. La documentación de prompting de Claude insiste en etiquetas estilo XML como delimitadores (capítulo siguiente). JSON frente a XML es elección del consumidor; el patrón de control (sobre claro + stop) es el mismo.

DOCS POR ENCIMA DEL JSON MODE DEL AULA

Cualquier interruptor «JSON mode» de una consola puede haberse movido. Priman las **salidas estructuradas** y la Messages API.

Documentación oficial

• Salidas estructuradas

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

JSON acotado por esquema en modelos soportados.

• Messages API

<https://docs.claude.com/en/api/messages>

Prefill de assistant y stop_sequences del laboratorio.

• Visión general de prompting

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Instrucciones de formato de salida y ejemplos.